Progetto S2-L5

```python
import datetime

while True
    comando_utente = input("Cosa vuoi sapere? ")
    if comando_utente == "esci":
        print("Arrivederci!")
    break
    else:
        print(assistente_virtuale(comando_utente))

def assistente_virtuale(comando):
    if comando == "Qual è la data di oggi?":
        oggi = datetime.datetoday()
        risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
    elif comando == "Che ore sono?":
        ora_attuale = datetime.datetime.now().time()
        risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
    elif comando == "Come ti chiami?":
        risposta = "Mi chiamo Assistente Virtuale"
    else:
        risposta = "Non ho capito la tua domanda."
    return risposta
```

qui manca : alla fine del True

Indentazione scorretta: l'else non può venire dopo un break fuori dall'if

La struttura è invertita. La funzione def dovrebbe essere definita prima del ciclo while.

Non esiste "datetoday" nel modulo "datetime"di Python*

*si dovrebbe sostituire con:

oggi = datetime.date.today()

(datetime è il modulo
date è la classe
today è il metodo della classe)

Traccia dell'esercizio

L'esercizio richiedeva di analizzare un programma Python senza eseguirlo, con l'obiettivo di comprenderne il funzionamento e individuare eventuali problemi presenti nel codice. In particolare, era richiesto di capire cosa fa il programma, individuare eventuali errori di sintassi o logici, riconoscere possibili casi non gestiti dal programma e proporre una soluzione per ciascun problema individuato. Tutta l'analisi è stata svolta leggendo direttamente il codice sorgente, senza eseguire il programma su Visual Studio Code.

Comprensione del funzionamento del programma

Osservando il codice è possibile capire che il programma vuole simulare un semplice assistente virtuale con cui l'utente può interagire tramite la tastiera. All'inizio viene importato il **modulo datetime**, che serve per lavorare con data e ora.

Successivamente il programma entra in un ciclo while che dovrebbe restare attivo finché l'utente non decide di uscire. All'interno del ciclo viene chiesto all'utente di inserire un comando tramite la funzione input. Se l'utente scrive "esci", il programma dovrebbe stampare un messaggio di saluto e terminare il ciclo. In tutti gli altri casi il comando inserito viene passato alla funzione assistente_virtuale, che ha il compito di analizzare la richiesta e restituire una risposta.

La funzione contiene una serie di condizioni if ed elif che controllano il contenuto della domanda dell'utente. Se l'utente chiede la data, il programma restituisce la data corrente. Se invece chiede l'ora, il programma restituisce l'orario attuale. È prevista anche una risposta nel caso in cui l'utente chieda il nome dell'assistente. Se la domanda non corrisponde a nessuna di queste condizioni, il programma restituisce un messaggio che indica che la richiesta non è stata compresa.

Errori individuati nel codice

Analizzando il codice riga per riga è possibile individuare alcuni errori che impedirebbero al programma di funzionare correttamente:

Il primo errore si trova nella riga in cui viene dichiarato il ciclo **while True** (*riga 3*). In Python, tutte le strutture di controllo devono terminare con i due punti. Nel codice analizzato questi due punti mancano, quindi il programma genererebbe un errore di sintassi. Per correggere questo problema è sufficiente aggiungere i due punti alla fine della riga.

Un secondo problema riguarda l'istruzione **break** (*riga 7*). Nel codice originale il break non è indentato allo stesso livello delle istruzioni contenute nell'**if**. In Python l'indentazione è fondamentale perché definisce i blocchi di codice. In questo caso il **break** risulta esterno alla condizione **if**, e verrebbe quindi eseguito sempre. Questo significherebbe che il ciclo while terminerebbe subito dopo il primo input dell'utente, invece di continuare finché l'utente non scrive "esci". La soluzione è spostare il **break** all'interno del blocco if.
Collegato a questo problema c'è anche l'errore nella struttura **else**(*riga 8*). In Python l'else deve essere direttamente collegato all'if corrispondente e avere la stessa indentazione. Nel codice originale, però, tra if ed else è presente l'istruzione break, che interrompe la struttura della condizione e genera un errore di sintassi. Sistemando l'indentazione del break, anche questo problema viene risolto.
Un altro errore si trova nella riga in cui si cerca di ottenere la data corrente usando **datetime.datetoday()** (*riga 13*). Questa funzione non esiste nel modulo **datetime** nella documentazione Python (www.doc.python.org). Il metodo corretto per ottenere la data corrente è **datetime.date.today()**. Se il programma venisse eseguito nella forma originale, Python genererebbe un errore perché non troverebbe l'attributo richiesto nella libreria.

Un'ultima osservazione riguarda la posizione della **funzione assistente_virtuale**. Nel codice originale la funzione è definita dopo il ciclo while in cui viene utilizzata. Dal momento che Python esegue il codice dall'alto verso il basso, è preferibile definire le funzioni prima del punto in cui vengono chiamate. Spostare la definizione della funzione prima del ciclo rende il codice più chiaro ed evita possibili problemi di esecuzione.

Situazioni non gestite dal programma

Oltre agli errori veri e propri, il programma presenta anche alcune limitazioni nella gestione delle richieste dell'utente. Le condizioni all'interno della funzione controllano infatti frasi molto specifiche. Questo significa che anche piccole variazioni nella scrittura possono impedire al programma di riconoscere il comando.

Ad esempio, il programma distingue tra lettere maiuscole e minuscole. Se l'utente scrive la stessa domanda ma con una capitalizzazione diversa, la richiesta non verrà riconosciuta. Lo stesso problema può verificarsi se l'utente inserisce spazi in più all'inizio o alla fine della frase (queste due problematiche si potrebbero risolvere inserendo **comando = comando.lower().strip()** subito dopo **def assistente_virtuale(comando):** e stessa cosa andrebbe aggiunta all'input dell'utente in caso digitasse "esci" dopo il while modificando così
if comando_utente.lower().strip() == "esci":). Inoltre, il programma riconosce solo una formulazione precisa delle domande: richieste simili ma scritte in modo leggermente diverso non vengono interpretate.

Queste situazioni non impediscono al programma di funzionare, ma rappresentano comunque casi non gestiti che potrebbero essere migliorati per rendere l'assistente più flessibile.

Conclusione

L'analisi del codice ha permesso di capire il funzionamento generale del programma e di individuare diversi problemi semplicemente leggendo il codice sorgente, senza bisogno di eseguirlo. Sono stati individuati errori di sintassi, problemi legati all'indentazione, un utilizzo non corretto di una funzione della libreria datetime e alcune limitazioni nella gestione degli input dell'utente.

Per ciascun errore è stata proposta una possibile correzione, con l'obiettivo di rendere il programma funzionante e più robusto. Questo tipo di analisi dimostra come una lettura attenta del codice possa essere sufficiente per individuare molti problemi prima ancora di eseguire il programma.

Codice corretto:

```
import datetime

def assistente_virtuale(comando):
    comando = comando.lower().strip()

    if comando == "Qual è la data di oggi?":
        oggi = datetime.date.today()
        risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
    elif comando == "Che ore sono?":
        ora_attuale = datetime.datetime.now()
        risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")

    elif comando == "come ti chiami?":
        risposta = "Mi chiamo Assistente Virtuale"

    else:
        risposta = "Non ho capito la tua domanda."

    return risposta


while True:

    comando_utente = input("Cosa vuoi sapere? ")

    if comando_utente.lower().strip() == "esci":
        print("Arrivederci!")
        break

    else:
        print(assistente_virtuale(comando_utente))
```